



Tratamiento de datos y señales analógicas en STEP 7

Sistemas secuenciales
programables



Índice



- 9.1. Tipos de datos en el lenguaje STEP 7**
- 9.2. Tipos de datos simples**
 - 9.2.1. Organización de los tipos de datos simples
- 9.3. Notación de números en STEP 7**
- 9.4. Operación de transferencia (MOVE)**
 - 9.4.1. Transferencia de datos con un formato determinado
 - 9.4.2. Operaciones de transferencia en el GRAFCET
- 9.5. Operaciones matemáticas básicas**
- 9.6. Tratamiento de señales analógicas**
 - 9.6.1. ¿Qué debemos conocer para trabajar con señales analógicas?
 - 9.6.2. Lectura de señales analógicas de entrada
 - 9.6.3. Escritura de señales analógicas de salida
 - 9.6.4. Normalización y escalado de señales analógicas



Introducción

Ya hemos estado hablando en los temas anteriores de la programación y estructura básica del STEP 7, en este tema nos centraremos en que tipo de elementos y numeración usaremos para que sean compatibles con las instrucciones del lenguaje KOP.

Al finalizar esta unidad

- + Identificaremos diferentes tipos de datos en STEP 7
- + Podremos hacer bloques MOVE de transferencia de datos
- + Conoceremos algunas señales analógicas
- + Usaremos instrucciones de transferencia en las acciones de un GRAFCET

9.1.

Tipos de datos en el lenguaje STEP 7

Para realizar diferentes operaciones tales como tratar señales, comparar valores, etc. los procesos industriales tratarán otros tipos de datos, además de los booleanos, para poder procesar magnitudes físicas.

Los tipos de datos que nos ofrece STEP 7 serán de tipo simple (almacenan números, cadenas de caracteres y booleanos) y compuestos (estructuras de variables).



9.2.

Tipos de datos simples

Podremos almacenar los datos numéricos (enteros y reales), caracteres y booleanos. Los diferentes tipos de datos simples tendrán un tamaño en bits que permitirán determinar el rango del valor. Se definirán con un nombre y la zona de memoria del PLC. El tamaño lo definiremos con la potencia de $2^{n_{bits}}$

Tipos de datos				
Tipo de dato	Tamaño	Descripción	Rango	Ejemplo de direccionamiento
BOOL	1 bit	Valor booleano	1-0	I0.0 Q0.1 M8.3
BYTE	8 bits	Byte sin signo	0-255	I0 QB1 MB12
WORD	16 bits	Entero sin signo	0 a 65535	I0W0 QW1 MW12
DWORD	32 bits	Doble entero sin signo	0 a 4294967295	I0D0 QD1 MD12
DINT	32 bits	Doble entero con signo	-2147483648 hasta 2147483647	I0D0 QD1 MD12
REAL	32 bits	Valor de 32 bits en coma flotante	1175495 e-38 hasta 3402823 e38	MD12
S5TIME	16 bits	Tiempo SIMATIC	S5T#0H_0M_0S_10MS hasta T#24D_20H_31M_23S_647MS	MD10
TIME	32 bits	Tiempo IEC	T#24D_20H_31M_23S_648MS hasta T#24D_20H_31M_23S_647MS	MD15
CHAR	8 bits	Carácter	A,B,C,etc.	MB20

9.2.1. Organización de los tipos de datos simples

Cuando tengamos variables que superen un bit, el direccionamiento absoluto de memoria del PLC la haremos con un byte de referencia. A continuación, veremos una estructura en binario de como formar una doble palabra con tipos de datos simples:

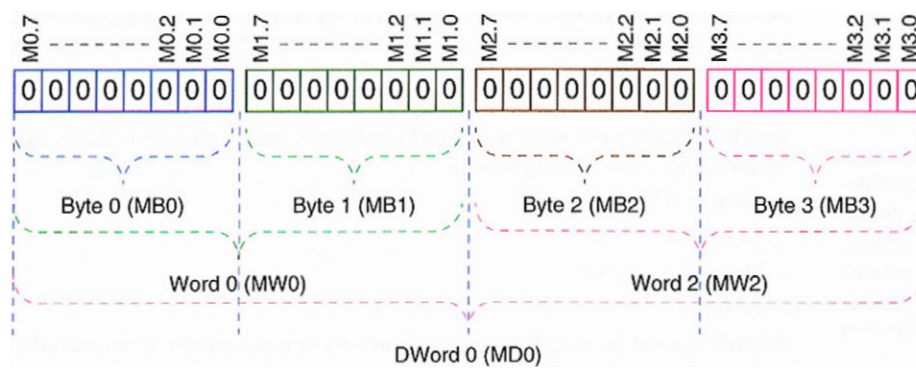


Imagen 1. Formato de tipos de datos simples con una doble palabra

9.3.

Notación de números en STEP 7

Podremos escribir los números en STEP 7 de muchas maneras diferentes, aunque la mayoría tendrá un sistema de numeración decimal. Si queremos escribir otro sistema de numeración deberemos hacerlo como: 2# (binario), 8# (octal), 16# (hexadecimal) y C# (BCD).

9.4.

Operación de transferencia (MOVE)

Usaremos el MOVE para enviar datos a la zona de memoria que nosotros deseemos del PLC. En KOP, tendrá forma de caja con unas entradas y salidas. Las entradas son EN (habilitara el bloque) e IN (escribe el dato de transferencia). Mientras que las salidas serán ENO (habilitara otros bloques en cascada) y OUT (zona de memoria donde recibir el dato).

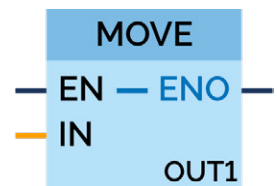


Imagen 2. Operador MOVE

9.4.1. Transferencia de datos con un formato determinado

Tendremos que transferir datos con un determinado formato y este deberá tener el tamaño y formato correcto. En el segmento que vemos, la entrada PT no escribiremos un valor fijo, ya que buscamos que poder cambiar el tiempo de forma dinámica.

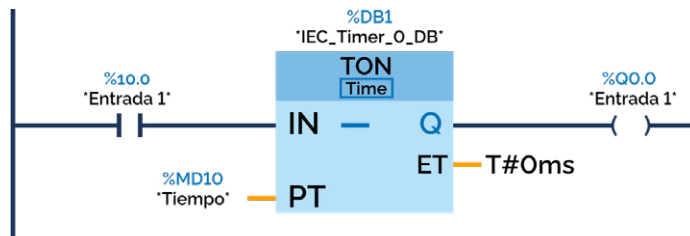


Imagen 3. Tiempo dinámico en entrada PT de un temporizador

Tendremos que crear todos los bloques MOVE que queramos configurar en cualquier temporizador. Aquí crearemos dos temporizadores, uno de 3 y otro de 8 segundos, donde uno estará abierto y otro cerrado.

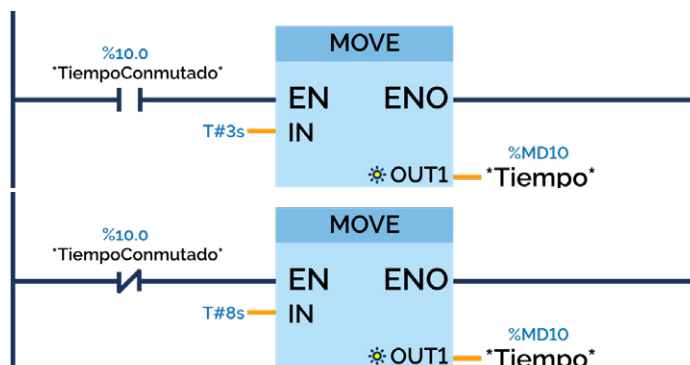


Imagen 4. Asignar valores de tiempo a una variable

9.4.2. Operaciones de transferencia en el GRAFCET

Cuando usemos el GRAFCET con este tipo de operaciones, tendremos una zona denominada de acciones donde no será necesario realizar hacer nada mas en otras zonas de programación.

Cuando usemos (:=) en el GRAFCET, supondrá que en la zona de acción tendremos un operador de transferencia MOVE.

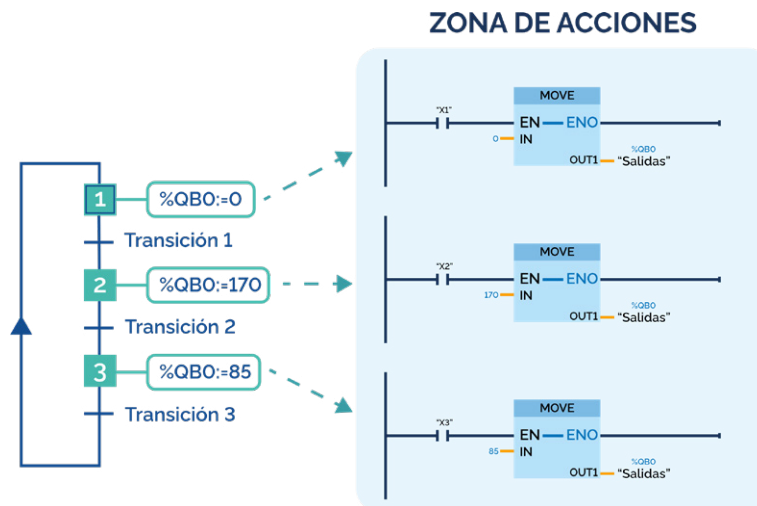
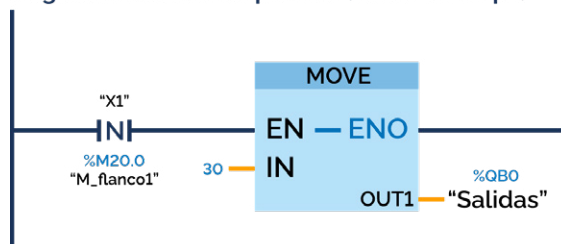


Imagen 5. Acciones con operaciones de transferencia

Cuando hayamos hecho la zona de acciones con el MOVE, la variable que se le asignara, aunque el bloque se desactive, o lo que es lo mismo, no es obligatorio mantener un 1 en la entrada.

Cuando tengamos una acción por flanco positivo realizaremos la transferencia cuando entre a la etapa y el negativo cuando salga de ella.

Asignación con flanco positivo (entrar en etapa)



Asignación con flanco negativo (Salir en etapa)

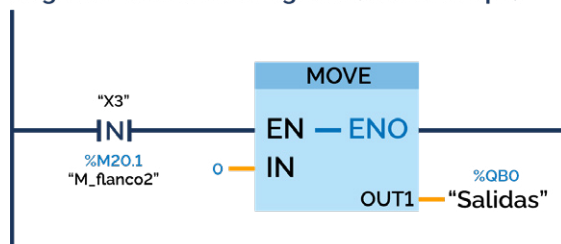


Imagen 6. Asignaciones condicionadas a flancos



9.5.

Operaciones matemáticas básicas

Podremos realizar diversas operaciones de cálculo con los PLC ya que disponen de una gran potencia. Describiremos algunas de las operaciones básicas de cálculo tales como suma, resta, multiplicación y división.

Hablaremos del multiplicador, pero será igual para todos los operadores.

- > Arriba tendremos el tipo de operación
- > A la izquierda tendremos las entradas IN, las cuales tendrán los números con los que operamos.
- > En la derecha tendremos el OUT con el resultado de la operación

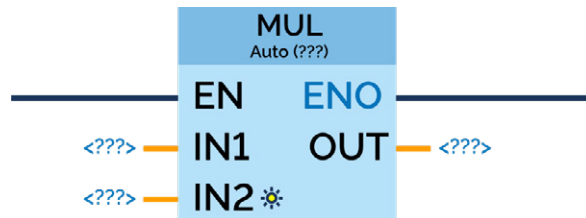


Ilustración 7. Bloque de operación matemática

9.6.

Tratamiento de señales analógicas

9.6.1. ¿Qué debemos conocer para trabajar con señales analógicas?

Tipo de señal

Tendremos que saber el tipo de señal que se admite cuando vayamos a trabajar en la conexión de un hardware. Debemos conocer si la tensión o intensidad es unipolar (valores positivos) o bipolar (negativos y positivos).

Valor del dato analógico

Estos valores se guardarán en una zona de memoria de 16 bits. Como hemos comentado anteriormente los valores son positivos o negativos. Si son unipolares serán positivos y el rango nominal es de 0 a 27648, si son bipolares serán negativos y positivos y su rango nominal será de -27648 a 27648.

El direccionamiento físico al canal analógico

Será la zona de memoria donde tendremos que apuntar si queremos trabajar con esa señal. Los datos ocuparán dos bits (16 bits). Se harán de dos maneras:

- > **Direccionamiento a la periferia:** la sintaxis será PIWxxx en señales analógicas y en señales digitales PQWxxx, y xxx será el número del canal analógico.
- > **Direccionamiento a zonas de memoria:** apuntan al byte de referencia, diciéndote el tamaño en memoria de esta zona. Para señales analógicas de entrada %IWxxx y salida %QWxxx.

9.6.2. Lectura de señales analógicas de entrada

La lectura de esta será de %IWxxx, donde xxx deberá ser el primer byte de ese canal. La recomendación que se da es que cuando tengamos una variable INT es que se pase a una entrada analógica para usarla con las operaciones.

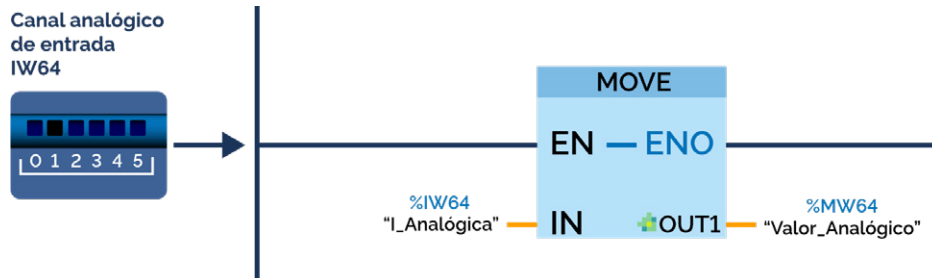


Imagen 8. Transferencia del valor de una entrada analógica a una variable

9.6.3. Escritura de señales analógicas de salida

Con transferir un valor numérico entre 0 y 27648 podremos escribir un dato en cualquier salida analógica. Como ya comentamos anteriormente será aconsejable cambiar el formato INT. La sintaxis será %QWxxx, donde xxx deberá ser el primer byte de ese canal.

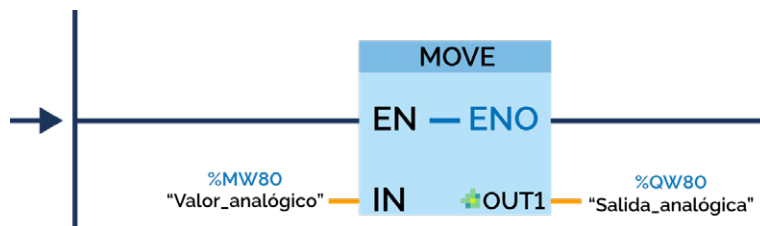


Imagen 9. Transferencia a una salida analógica desde una variable

9.6.4. Normalización y escalado de señales analógicas

Cuando vayamos a trabajar dentro del rango de valores podemos llegar a confundirnos muy fácilmente si lo queremos relacionar con magnitudes físicas. Por lo que STEP 7 dispondrá de algunas instrucciones de programación que relacionan magnitudes con señales.

- > **NORM_X**: Normalizará un valor que reciba entre un máximo y mínimo, cuando procese dicho valor la salida será en formato coma flotante.
- > **SCALE_X**: Un número en coma flotante escalará el valor para ajustarlo entre un rango máximo y mínimo de valores y ese valor será una variable en formato INT.

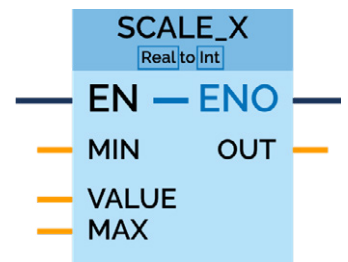


Imagen 10. SCALE_X

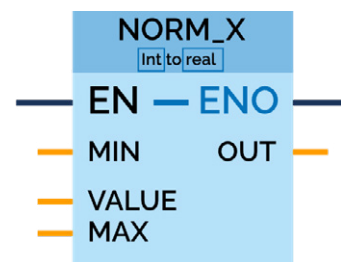


Imagen 11. NORM_X



 www.universae.com

